Module 07 Lab - Better Model Evaluation**Objective:** To move beyond simple accuracy and learn how to use more sophisticated and reliable evaluation techniques, including the confusion matrix, precision, recall, and cross-validation.**In this lab, you will write the code to generate and interpret these advanced evaluation metrics.**

Part 1: Setup and Model TrainingLet's first train a model so we have something to evaluate. We will use the Titanic dataset again to predict survival.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load and prepare data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets

df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Age'].fillna(df['Age'].median(), inplace=True)

features = ['Age', 'Pclass', 'Sex', 'Fare']
target = 'Survived'

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train a model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
/tmp/ipykernel_5557/543017654.py:9: FutureWarning: A value is trying to be se
The behavior will change in pandas 3.0. This inplace method will never work b

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.


  df['Age'].fillna(df['Age'].median(), inplace=True)
```

Part 2: The Confusion Matrix**Concept:** A confusion matrix gives you a more detailed breakdown of a model's performance than accuracy alone. It's a table that shows you where your model got things right and where it got them wrong.It has four quadrants:* **True Positives (TP):** Correctly predicted positive (e.g., predicted survival, and they did survive).* **True Negatives (TN):** Correctly predicted negative (e.g., predicted did not survive, and they did not).* **False Positives (FP):** Incorrectly predicted positive (e.g., predicted survival, but they did not). Also called a "Type I Error".* **False Negatives (FN):** Incorrectly predicted negative (e.g., predicted did not survive, but they did). Also called a "Type II Error".

Task 1: Generate and Plot a Confusion Matrix**Your Task:** Use `confusion_matrix` from `sklearn.metrics` to calculate the matrix and `seaborn.heatmap` to visualize it.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

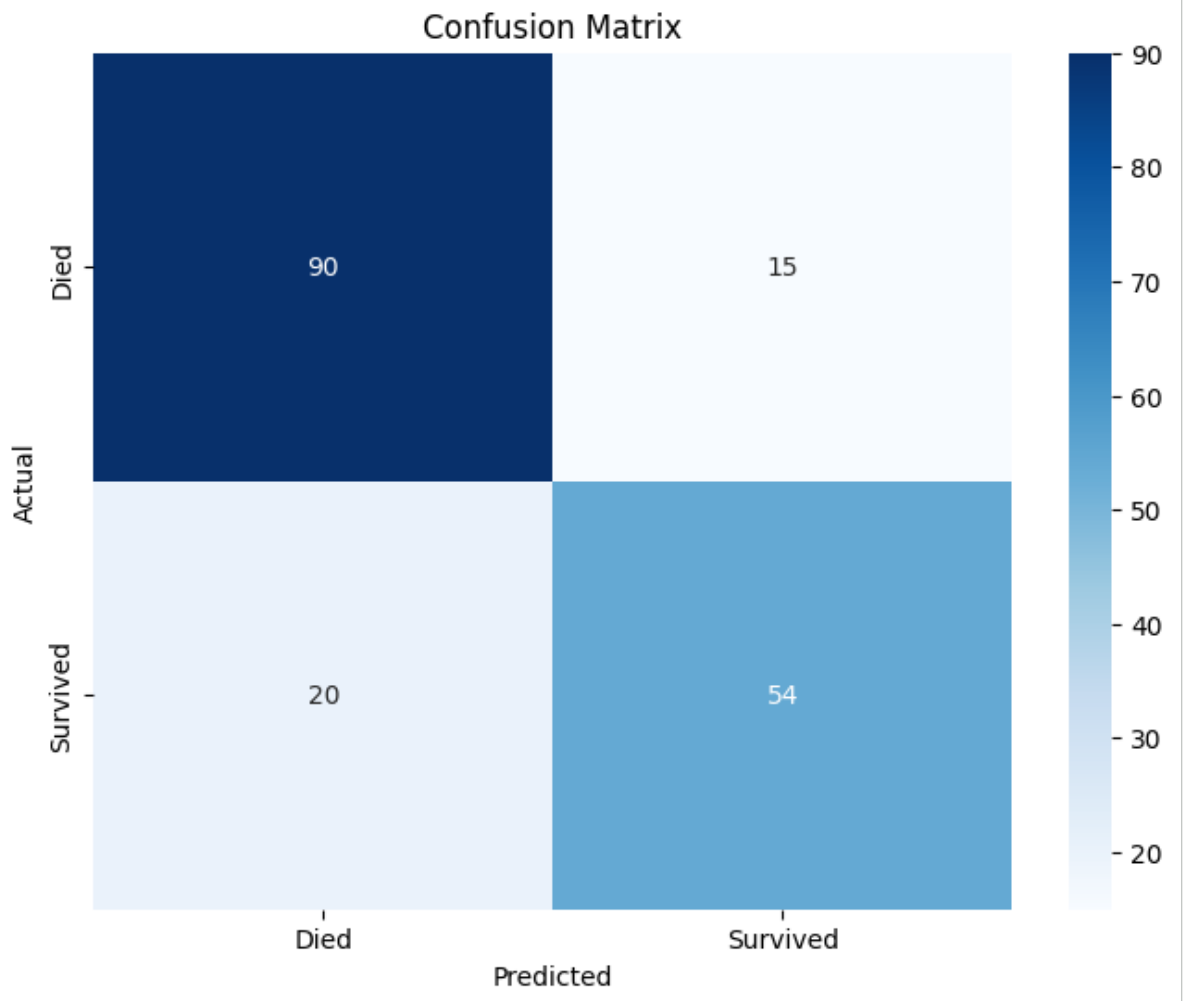
```
# --- ENTER YOUR CODE HERE ---

# 1. Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# 2. Create a heatmap to visualize it
plt.figure(figsize=(8, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Died', 'Survived'],
    yticklabels=['Died', 'Survived']
)

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Part 3: Precision, Recall, and F1-Score**Concept:** From the confusion matrix, we can calculate more nuanced metrics:* **Precision:** Of all the times the model predicted **positive**, how often was it correct? * Formula: `TP / (TP + FP)` * *Use Case:* When the cost of a **False Positive** is high. (e.g., a spam filter; you don't want to incorrectly mark an important email as spam).* **Recall (Sensitivity):** Of all the actual **positives**, how many did the model correctly identify? * Formula: `TP / (TP + FN)` * *Use Case:* When the cost of a **False Negative** is high. (e.g., a medical test for a serious disease; you don't want to miss a real case).* **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both.

Task 2: Generate a Classification Report**Your Task:** Use `classification_report` from `sklearn.metrics` to get a summary of these metrics for each class.

```
from sklearn.metrics import classification_report

# --- ENTER YOUR CODE HERE ---

# 1. Generate and print the classification report
report = classification_report(y_test, y_pred)
print(report)
```

```
              precision    recall  f1-score   support

           0       0.82      0.86      0.84       105
           1       0.78      0.73      0.76        74
```

```
              precision    recall  f1-score   support

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.80       179
weighted avg       0.80      0.80      0.80       179
```

Part 4: Cross-Validation**Concept:** A single train-test split can be lucky or unlucky. What if, by chance, all the "easy" examples ended up in our test set? Our accuracy score would be misleadingly high.**Cross-Validation (CV)** solves this. It splits the data into multiple "folds" (e.g., 5 or 10). It then trains and evaluates the model multiple times, using a different fold as the test set each time. The final score is the average of the scores from all folds.This gives a much more robust and reliable estimate of the model's true performance.

Task 3: Perform Cross-Validation**Your Task:** Use `cross_val_score` from `sklearn.model_selection` to perform 5-fold cross-validation on your model.

```
from sklearn.model_selection import cross_val_score

# --- ENTER YOUR CODE HERE ---

# 1. Perform 5-fold cross-validation
# We use the entire dataset (X, y), as cross_val_score handles the splitting i
cv_scores = cross_val_score(model, X, y, cv=5)

# 2. Print the scores for each fold and the average score
print(f"Scores for each fold: {cv_scores}")
print(f"Average CV Score: {cv_scores.mean():.2%}")
print(f"Standard Deviation of CV Scores: {cv_scores.std():.4f}")
```

```
Scores for each fold: [0.7877095  0.78089888 0.78651685 0.7752809  0.80337079
Average CV Score: 78.68%
```

```
Standard Deviation of CV Scores: 0.0094
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **Describe a real-world scenario where you would care more about a model's Precision than its Recall.**2. **Describe a real-world scenario where you would care more about a model's Recall than its Precision.**3. ***Why is a cross-validation score generally more trustworthy than a score from a single train-test split?***[

1. **Describe a real-world scenario where you would care more about a model's Precision than its Recall.**

I would care more about precision when creating an email spam filter. I would want the model to be accurate when labeling emails as spam because I don't want important emails to be incorrectly marked as spam. It is better to let a few spam emails into the inbox than to accidentally hide an important message.

2. **Describe a real-world scenario where you would care more about a model's Recall than its Precision.**

   I would care more about recall when using a model to identify people who might have a serious disease. I would want the model to find as many possible cases as it can, even if it sometimes incorrectly identifies a healthy person as potentially sick. Missing a real case could prevent someone from getting the medical attention they need.

3. **Why is a cross-validation score generally more trustworthy than a score from a single train-test split?**

   Cross-validation is generally more trustworthy because it tests the model multiple times using different parts of the dataset. A single train-test split might give unusually good or bad results depending on which examples end up in the test set. Cross-validation averages the results from several tests, giving a more reliable idea of how well the model might perform on new data.]**

Module 07 Lab - Better Model Evaluation**Objective:**

To move beyond simple accuracy and learn how to use more sophisticated and reliable evaluation techniques, including the confusion matrix, precision, recall, and cross-validation.**In this lab, you will write the code to generate and interpret these advanced evaluation metrics.**

Part 1: Setup and Model TrainingLet's first train a model so we have something to evaluate. We will use the Titanic dataset again to predict survival.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load and prepare the Titanic dataset
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titani
df = pd.read_csv(url)

# Select features and target
features = ['Age', 'Pclass', 'Sex', 'Fare']
target = 'Survived'

# Prepare the data
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

X = df[features]
y = df[target]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Train the model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

print("Model trained successfully!")
print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
```

```
Model trained successfully!
Training rows: 712
Testing rows: 179
```

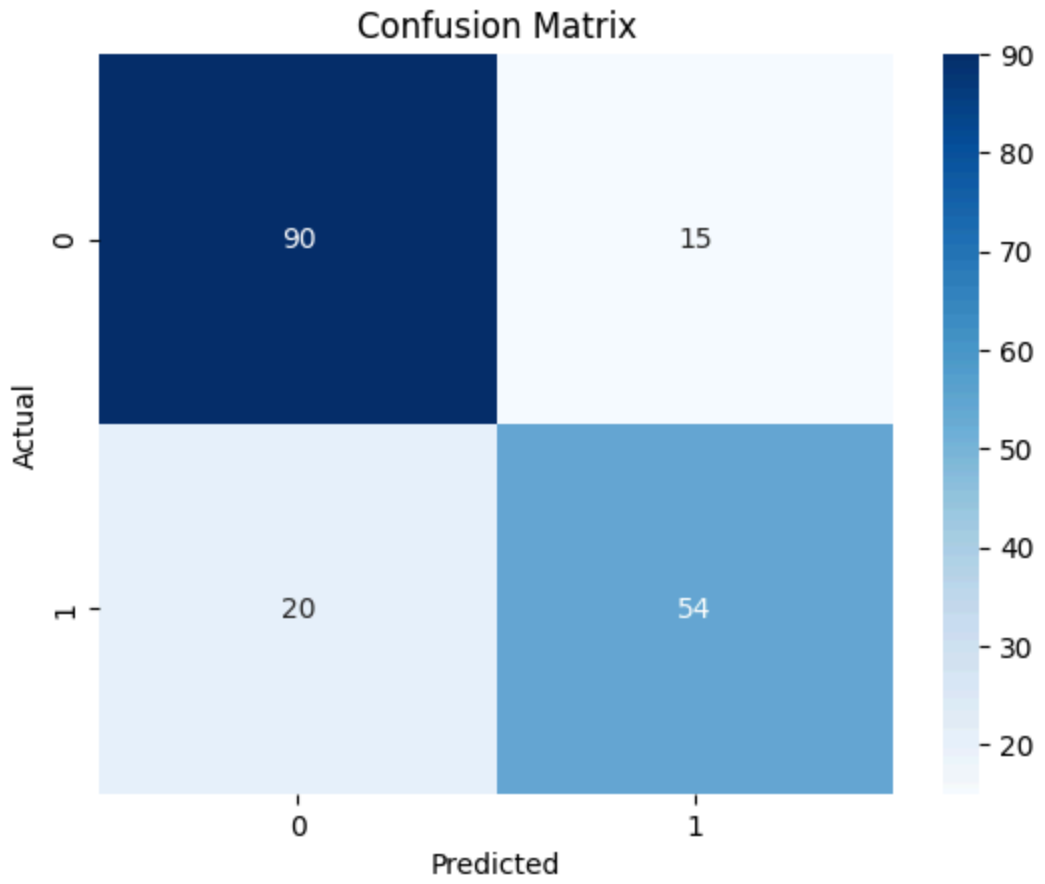
Part 2: The Confusion Matrix**Concept:** A confusion matrix gives you a more detailed breakdown of a model's performance than accuracy alone. It's a table that shows you where your model got things right and where it got them wrong.It has four quadrants:* **True Positives (TP):** Correctly predicted positive (e.g., predicted survival, and they did survive).* **True Negatives (TN):** Correctly predicted negative (e.g., predicted did not survive, and they did not).* **False Positives (FP):** Incorrectly predicted positive (e.g., predicted survival, but they did not). Also called a "Type I Error".* **False Negatives (FN):** Incorrectly predicted negative (e.g., predicted did not survive, but they did). Also called a "Type II Error".

Task 1: Generate and Plot a Confusion Matrix**Your Task:** Use `confusion_matrix` from `sklearn.metrics` to calculate the matrix and `seaborn.heatmap` to visualize it.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# 2. Plot the confusion matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Part 3: Precision, Recall, and F1-Score**Concept:** From the confusion matrix, we can calculate more nuanced metrics:* **Precision:** Of all the times the model predicted **positive**, how often was it correct? * Formula: `TP / (TP + FP)` * *Use Case:* When the cost of a **False Positive** is high. (e.g., a spam

filter; you don't want to incorrectly mark an important email as spam).* **Recall (Sensitivity):** Of all the actual **positives**, how many did the model correctly identify? * Formula: `TP / (TP + FN)` * *Use Case:* When the cost of a **False Negative** is high. (e.g., a medical test for a serious disease; you don't want to miss a real case).* **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both.

Task 2: Generate a Classification Report**Your Task:** Use `classification_report` from `sklearn.metrics` to get a summary of these metrics for each class.

```
from sklearn.metrics import classification_report

# Generate and print the classification report
report = classification_report(y_test, y_pred)
print(report)
```

```
              precision    recall  f1-score   support

           0       0.82      0.86      0.84       105
           1       0.78      0.73      0.76        74

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.80       179
weighted avg       0.80      0.80      0.80       179
```

Part 4: Cross-Validation**Concept:** A single train-test split can be lucky or unlucky. What if, by chance, all the "easy" examples ended up in our test set? Our accuracy score would be misleadingly high.**Cross-Validation (CV)** solves this. It splits the data into multiple "folds" (e.g., 5 or 10). It then

trains and evaluates the model multiple times, using a different fold as the test set each time. The final score is the average of the scores from all folds.This gives a much more robust and reliable estimate of the model's true performance.

Task 3: Perform Cross-Validation**Your Task:** Use `cross_val_score` from `sklearn.model_selection` to perform 5-fold cross-validation on your model.

```
from sklearn.model_selection import cross_val_score# --- ENTER YOUR CODE HERE -
cv_scores = cross_val_score(model, X, y, cv=5)

# 2. Display the score from each fold
print("Scores for each fold:", cv_scores)

# 3. Calculate and display the average score
print("Average CV Score:", round(cv_scores.mean(), 4))

# 4. Display the standard deviation
print("Standard Deviation:", round(cv_scores.std(), 4))
```

```
Scores for each fold: [0.7877095  0.78089888 0.78651685 0.7752809  0.80337079]
Average CV Score: 0.7868
Standard Deviation: 0.0094
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.

1. Describe a real-world scenario where you would care more about Precision than Recall.

Precision is more important in a situation where false positive predictions are costly. For example, in an email spam filter, I would want high precision so that emails marked as spam are actually spam. This reduces the risk of incorrectly sending important emails to the spam folder, even if some spam emails are missed. Also in legal instance, the maxim "innocent until proven guilty" hereby convicting an innocent person (false positive).

2. Describe a real-world scenario where you would care more about Recall than Precision.

Recall is more important when missing a real positive case could have serious consequences. For example, in medical screening for a serious disease, I would want high recall so that the model identifies as many people who may have the disease as possible. Some false positives may be acceptable because further medical testing can confirm the results. Also in banking, having to flag credit card transactions that look like identity theft or unauthorized purchases.

3. Why is a cross-validation score generally more trustworthy than a score from a single train-test split?

Cross-validation is generally more trustworthy because the model is trained and evaluated across multiple different data splits. This reduces the risk that the evaluation result depends on one unusually easy or difficult test set. The average score provides a more reliable estimate of the model's performance on unseen data.

Module 07 Lab - Better Model Evaluation**Objective:** To move beyond simple accuracy and learn how to use more sophisticated and reliable evaluation techniques, including the confusion matrix, precision, recall, and cross-validation.**In this lab, you will write the code to generate and interpret these advanced evaluation metrics.**

Part 1: Setup and Model TrainingLet's first train a model so we have something to evaluate. We will use the Titanic dataset again to predict survival.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load and prepare data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Age'].fillna(df['Age'].median(), inplace=True)

features = ['Age', 'Pclass', 'Sex', 'Fare']
target = 'Survived'

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
/tmp/ipykernel_2836/676736891.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['Age'].fillna(df['Age'].median(), inplace=True)
```

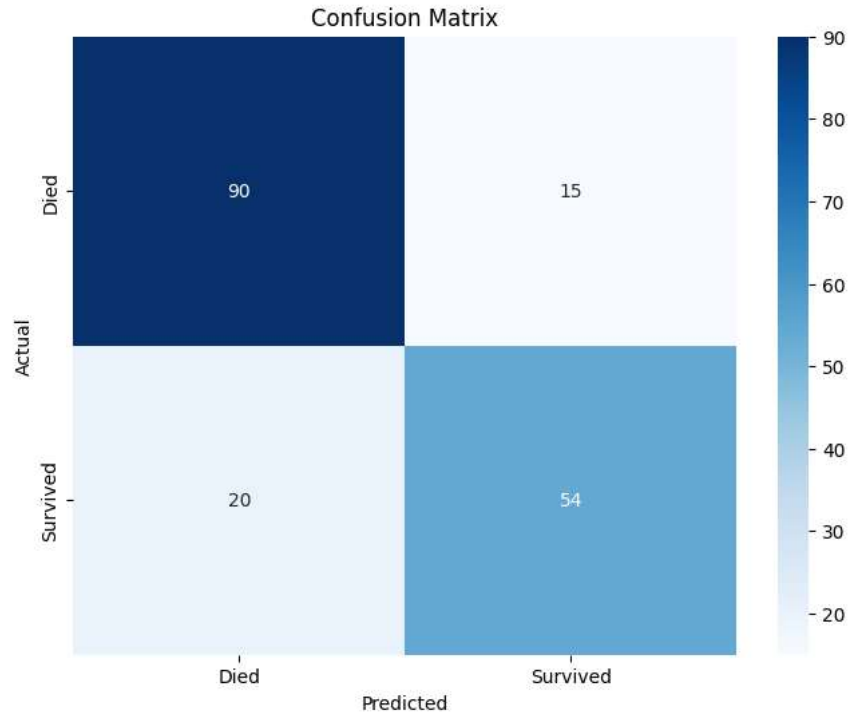
Part 2: The Confusion Matrix**Concept:** A confusion matrix gives you a more detailed breakdown of a model's performance than accuracy alone. It's a table that shows you where your model got things right and where it got them wrong.It has four quadrants:* **True Positives (TP):** Correctly predicted positive (e.g., predicted survival, and they did survive).* **True Negatives (TN):** Correctly predicted negative (e.g., predicted did not survive, and they did not).* **False Positives (FP):** Incorrectly predicted positive (e.g., predicted survival, but they did not). Also called a "Type I Error".* **False Negatives (FN):** Incorrectly predicted negative (e.g., predicted did not survive, but they did). Also called a "Type II Error".

Task 1: Generate and Plot a Confusion Matrix**Your Task:** Use `confusion_matrix` from `sklearn.metrics` to calculate the matrix and `seaborn.heatmap` to visualize it.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Died', 'Survived'],
    yticklabels=['Died', 'Survived']
)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Part 3: Precision, Recall, and F1-Score**Concept:** From the confusion matrix, we can calculate more nuanced metrics:* **Precision:** Of all the times the model predicted **positive**, how often was it correct? * Formula: `TP / (TP + FP)` * *Use Case:* When the cost of a **False Positive** is high. (e.g., a spam filter; you don't want to incorrectly mark an important email as spam).* **Recall (Sensitivity):** Of all the actual **positives**, how many did the model correctly identify? * Formula: `TP / (TP + FN)` * *Use Case:* When the cost of a **False Negative** is high. (e.g., a medical test for a serious disease; you don't want to miss a real case).* **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both.

Task 2: Generate a Classification Report**Your Task:** Use `classification_report` from `sklearn.metrics` to get a summary of these metrics for each class.

```
from sklearn.metrics import classification_report

report = classification_report(y_test, y_pred)
print(report)
```

```
              precision    recall  f1-score   support
```

```
              0       0.82      0.86      0.84       105
              1       0.78      0.73      0.76        74

       accuracy                           0.80       179
      macro avg       0.80      0.79      0.80       179
   weighted avg       0.80      0.80      0.80       179
```

Part 4: Cross-Validation**Concept:** A single train-test split can be lucky or unlucky. What if, by chance, all the "easy" examples ended up in our test set? Our accuracy score would be misleadingly high.**Cross-Validation (CV)** solves this. It splits the data into multiple "folds" (e.g., 5 or 10). It then trains and evaluates the model multiple times, using a different fold as the test set each time. The final score is the average of the scores from all folds.This gives a much more robust and reliable estimate of the model's true performance.

Task 3: Perform Cross-Validation**Your Task:** Use `cross_val_score` from `sklearn.model_selection` to perform 5-fold cross-validation on your model.

```
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(model, X, y, cv=5)

print(f"Scores for each fold: {cv_scores}")
print(f"Average CV Score: {cv_scores.mean():.2%}")
print(f"Standard Deviation of CV Scores: {cv_scores.std():.4f}")
```

```
Scores for each fold: [0.7877095  0.78089888 0.78651685 0.7752809  0.80337079]
Average CV Score: 78.68%
Standard Deviation of CV Scores: 0.0094
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **Describe a real-world scenario where you would care more about a model's Precision than its Recall.**2. **Describe a real-world scenario where you would care more about a model's Recall than its Precision.**3. **Why is a cross-validation score generally more trustworthy than a score from a single train-test split?

Describe a real-world scenario where you would care more about a model's Precision than its Recall.

Financial Fraud Detection: When a model marks a transaction as fraud, the bank may freeze the customer's credit card. If a normal transaction is wrongly marked as fraud, it can cause problems for the customer. High precision is important because most transactions marked as fraud should actually be fraud.

Describe a real-world scenario where you would care more about a model's Recall than its Precision.

Medical Diagnosis: When checking for a serious disease such as cancer, missing a patient who has the disease can be very dangerous. High recall is important because the model should find as many real cases as possible, even if it sometimes identifies healthy people as possible cases. These people can have more tests to confirm the results.

Why is a cross-validation score generally more trustworthy than a score from a single train-test split?

A single train test split depends on which data is used for testing. If the test data is too easy or too difficult, the score may not show the model's real performance. Cross validation tests the model on different parts of the dataset and calculates the average score. This gives a more reliable idea of how well the model performs.